

Rešavanje problema raspoređivanja poslova po mašinama metaheuristikama

Minimum Job Shop Scheduling

Nenad Pešić

septembar 2026.

Sadržaj

1	Uvod	2
2	Opis rešenja	3
3	Eksperimentalni rezultati	4
3.1	Okruženje za eksperimentisanje	4
3.2	Test instance	4
3.3	Metodologija	5
3.3.1	Budžet umesto vremena	5
3.3.2	Broj ponavljanja	6
3.3.3	Stohastičnost i reproducibilnost	6
3.4	Rezultati testiranja	7
3.5	Egzaktno rešavanje	8
4	Zaključak	12
5	Literatura	13

Poglavlje 1

Uvod

Poglavlje 2

Opis rešenja

Poglavlje 3

Eksperimentalni rezultati

Kako bismo olakšali kasniju interpretaciju rezultata, za početak ćemo dostaviti kontekstualne informacije o metodologiji usporedbe dve optimizacione metode, veličini eksperimenata i okruženju na kojima su eksperimenti pokretani.

3.1 Okruženje za eksperimentisanje

Sve eksperimentalne skripte izvršavane su na privatnom računaru, čije se najznačajnije karakteristike nalaze u sledećoj tabeli:

Tabela 3.1: Karakteristike računara na kome su izvršena sva merenja.

komponenta	vrednost
procesor	Intel Core Ultra 7 165U
frekvencija	3,71 GHz
broj jezgara	12 fizičkih / 14 logičkih
radna memorija	62,2 GB
operativni sistem	Ubuntu 24.04.4 LTS, 7.0.0-28-generic kernel, x86-64
standardna biblioteka	glibc 2.39
programski jezik	Python 3.12.3, GCC 13.3.0
egzaktni rešavač	IBM ILOG CPLEX 22.2.0.0
biblioteka za modelovanje	PuLP 3.3.2

Vredi napomenuti da su sve razvijene metaheuristike implementirane tako da se izvršavaju na **jednoj niti** bez mogućnosti paralelizacije. Egzaktni rešavač (CPLEX) pokretan je sa podrazumevanim brojem niti, dakle sa svih 14 logičkih jezgara. Zbog te razlike, metode je najpreciznije porediti na osnovu broja poziva funkcije dekodiranja, a ne po utrošenom vremenu. Dužina izvršavanja navedena je kao dodatni, orijentacioni podatak.

3.2 Test instance

Pri evaluaciji razvijenih metoda korišćeno je devet različitih test instanci. Od njih devet, osam je preuzeto iz zbirke OR-Library (Beasley 1990). Instance `ft06`, `ft10` i `ft20` potiču iz rada (Fis-

her i Thompson 1963), dok instance **1a01–1a05** dolaze iz (Lawrence 1984). Poslednja, deveta, instanca **mini3** kreirana je za potrebe ovog rada. Predstavlja relativno malu instancu čiji se optimum može pronaći algoritmom grube sile. Kao takva služila je najpre pri validaciji ispravnosti implementiranih metoda optimizacije. Pregled najvažnijih informacija o svim instancama dat je u narednoj tabeli.

Tabela 3.2: Test instance sa dimenzijama, donjim granicama i objavljenim optimalnim vrednostima.

instanca	$n \times m$	operacija	donja granica	optimum	izvor optimuma
mini3	3×3	9	10	11	iscrpna pretraga
ft06	6×6	36	47	55	(Fisher i Thompson 1963)
1a01	10×5	50	666	666	donja granica
1a02	10×5	50	635	655	(Lawrence 1984)
1a03	10×5	50	588	597	(Lawrence 1984)
1a04	10×5	50	537	590	(Lawrence 1984)
1a05	10×5	50	593	593	donja granica
ft10	10×10	100	655	930	(Fisher i Thompson 1963)
ft20	20×5	100	1119	1165	(Fisher i Thompson 1963)

Donja granica u tabeli 3.2 predstavlja veću od dve trivijalne granice: (1) najdužeg posla i (2) najopterećenije mašine. Način računanja detaljno je opisan u odeljku o donjim granicama. Kod instanci **1a01** i **1a05** ta granica se poklapa sa optimumom, pa svako rešenje koje je dostigne ujedno i **dokazuje** svoju optimalnost, bez potrebe za egzaktnim rešavačem.

Instance su izabrane tako da pokriju što širi opseg težine. Konkretno, **mini3** i **ft06** sve razvijene metode rešavaju do optimuma, pa služe kao provera ispravnosti. Instance **1a01–1a05** razdvajaju metode po pouzdanosti. **ft10** i **ft20** su, uprkos skromnim dimenzijama, poznato teške. Instanca **ft10** je ostala nerešena punih dvadeset šest godina nakon objavljivanja (Jain i Meeran 1999).

3.3 Metodologija

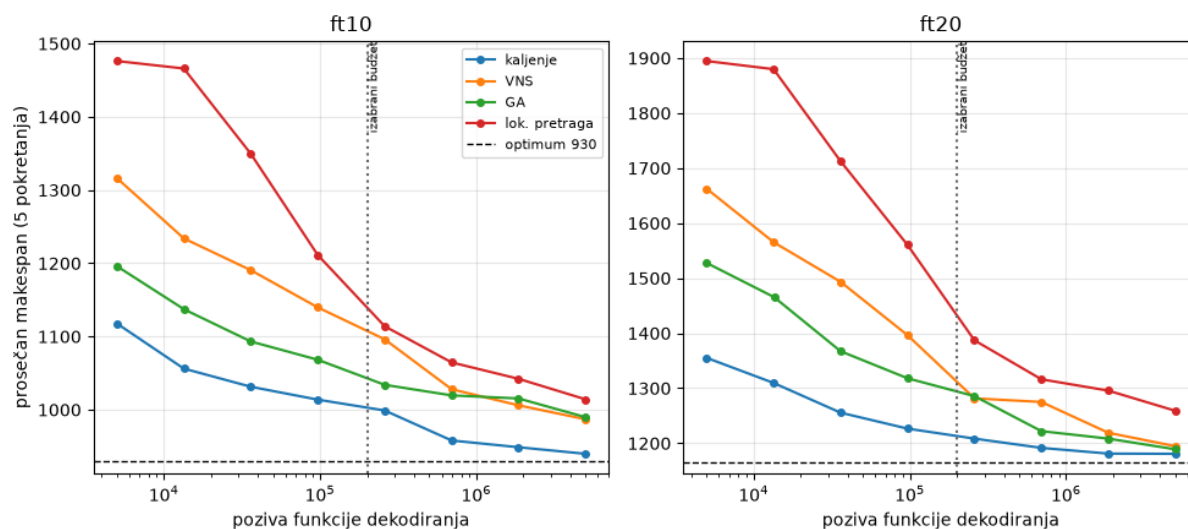
Za početak je potrebno objasniti način izbora veličine eksperimenta. Bilo je potrebno održati balans između validnosti rezultata i razumne upotrebe (pre svega vremenskih) resursa. Sa jedne strane, ne možemo dopustiti da eksperimente izvršavamo na premalom uzorku i time dovedemo u pitanje opštost zaključaka, dok je sa druge strane potrebno da se eksperimenti izvrše u razumnom vremenskom roku.

3.3.1 Budžet umesto vremena

Kao što je ranije navedeno, zbog nedostatka konkurentnosti u implementacijama algoritama, efikasnost različitih optimizacionih metoda nije pogodno upoređivati sa egzaktnim rešavačem na osnovu vremena izvršavanja. Kod međusobnog poređenja situacija je nešto bolja, ali se pokazalo

veoma izazovno unapred dozvoliti funkciji određeni vremenski okvir¹, pa je vreme izvršavanja zauzelo ulogu *retrospektivne mere*, dok su sami algoritmi unapred ograničeni brojem poziva funkcije dekodiranja.

Svakom algoritmu je unapred data gornja granica broja poziva ove funkcije i taj broj nazivamo *budžetom*. Kao što možemo videti na slici 3.1, prosečan rezultat nastavlja blago da se popravlja i nakon izabrane granice. Međutim, poredak metoda ostaje nepromenjen a preciznije apsolutne vrednosti zahtevaju nesrazmerno veći budžet. S tim na umu, izabrani budžet predstavlja kompromis između kvaliteta rešenja i ukupnog trajanje eksperimenta. Primećujemo da se vreme izvršavanja optimizacionog metoda razlikuje i pri istom budžetu, a razlog tome nalazi se u specifičnostima samih algoritama, gde se vreme „rasipa“ na pozive drugih pomoćnih metoda.



Slika 3.1: Prosečna vrednost funkcije cilja u zavisnosti od dodeljenog budžeta, za dve najteže instance. Obe ose prikazane su za pet nezavisnih pokretanja po tački. Uspravna isprekidana linija označava budžet usvojen u ostatku rada.

3.3.2 Broj ponavljanja

Po uzoru na literaturu o metaheuristikama, usvojen je izbor od **30 nezavisnih pokretanja** svake metode nad svakom instancom. Uz devet instanci i četiri različite metode dolazimo do ukupno **1080 pokretanja**. Ovo predstavlja dovoljno veliki uzorak za smisleni prikaz različitih osobina dobijenih rezultata, najpre: prosek, standardnu devijaciju, procenat dostizanja optimuma i slične.

3.3.3 Stohastičnost i reproducibilnost

Svako pokretanje optimizacionog problema u sebi sadrži stohastičnost u vidu **nasumičnog izbora početnog rešenja**. Pored toga, sve četiri metode na stohastičnost nailaze i u drugim delovima svoje implementacije, što nas navodi da je potrebno detaljnije obraditi pitanje **reproducibilnosti** rezultata.

Konkretno, postavljanjem iste početne vrednosti za generator pseudonasumičnih brojeva (engl. *seed*) omogućena je potpuna reproducibilnost svih dobijenih rezultata. Ponovno pokretanje pri-

¹Pre svega izazovnost se ogleda u činjenici da isti vremenski okvir na istoj mašini u različitim metodama podrazumeva različit broj koraka.

loženog koda dovodi do identičnih rezultata. Takođe *seed* vrednost sačuvana je u priloženim datotekama pa je moguće reprodukovati, ne samo eksperiment, već i jedno zasebno pokretanje optimizacione metode.

3.4 Rezultati testiranja

Dobijeni rezultati prikazani su kroz tri naredne tabele. U prvoj tabeli nalazi se **najbolje pronađeno rešenje** kroz **svih 30 pokretanja**. U drugoj tabeli vidimo prosek i standardnu devijaciju, dok treća tabela nosi informaciju o broju pokretanja u kojima je optimum dostignut.

U prvoj tabeli jasno je izražena informacija o mogućnosti (potencijalu) metode da pronađe najbolje rešenje. Kroz drugu i treću tabelu se ogleda pouzdanost različitih metoda pri višestrukoj upotrebi.

Tabela 3.3: Najbolje pronađeno rešenje kroz 30 pokretanja. Podebljane vrednosti poklapaju se sa objavljenim optimumom.

instanca	optimum	kaljenje	VNS	GA	lok. pretraga
mini3	11	11	11	11	11
ft06	55	55	55	55	55
la01	666	666	666	666	666
la02	655	655	655	655	663
la03	597	597	597	604	597
la04	590	590	590	590	594
la05	593	593	593	593	593
ft10	930	937	971	961	1002
ft20	1165	1173	1192	1180	1249

Tabela 3.4: Prosečna vrednost funkcije cilja kroz 30 pokretanja, sa standardnom devijacijom.

instanca	kaljenje	VNS	GA	lok. pretraga
mini3	11,0 ± 0,0	11,0 ± 0,0	11,0 ± 0,0	11,0 ± 0,0
ft06	55,0 ± 0,0	55,0 ± 0,0	55,0 ± 0,0	55,0 ± 0,0
la01	666,0 ± 0,0	666,0 ± 0,0	670,9 ± 8,9	666,0 ± 0,0
la02	659,0 ± 5,7	660,7 ± 6,6	669,9 ± 9,4	674,5 ± 9,0
la03	606,2 ± 7,4	607,6 ± 7,0	625,6 ± 14,2	623,9 ± 9,9
la04	591,2 ± 2,0	594,1 ± 4,2	606,0 ± 14,2	607,5 ± 7,3
la05	593,0 ± 0,0	593,0 ± 0,0	593,0 ± 0,0	593,0 ± 0,0
ft10	965,2 ± 16,3	1023,8 ± 24,7	1012,1 ± 30,9	1059,6 ± 30,0
ft20	1183,8 ± 11,1	1245,8 ± 29,9	1221,0 ± 26,6	1320,1 ± 40,9

Tabela 3.5: Broj pokretanja, od ukupno 30, u kojima je dostignut poznati optimum.

instanca	kaljenje	VNS	GA	lok. pretraga
mini3	30	30	30	30

instanca	kaljenje	VNS	GA	lok. pretraga
ft06	30	30	30	30
1a01	30	30	20	30
1a02	19	16	2	0
1a03	5	3	0	1
1a04	21	11	4	0
1a05	30	30	30	30
ft10	0	0	0	0
ft20	0	0	0	0

Tabela 3.3 pokazuje da instance `mini3`, `ft06` i `1a05` sve tri metode rešavaju do optimuma. Standardna devijacija na njima je nula (tabela 3.4), pa te instance ne razdvajaju metode i služe isključivo kao provera ispravnosti implementacije.

Poređenje metoda isključivo po najboljem pronađenom rešenju prikriva stvarnu informaciju o upotrebljivosti metode. Na primer, pri rešavanju instance `1a03` sve metode, osim genetskog algoritma, dostižu optimum. Ipak, tabela 3.5 pokazuje da ga lokalna pretraga pronalazi **u samo jednom od trideset pokretanja**, dok simulirano kaljenje to uspeva pet puta. Zbog toga je pri poređenju metoda potrebno uzeti u obzir i njihovo generalno ponašanje. Nepouzdanost je ograničiti se samo na najbolje primere.

Na instancama `1a02`, `1a03` i `1a04` razlike su najizraženije. Kaljenje dostiže optimum redom 19, 5 i 21 put, promenljive okoline 16, 3 i 11 puta, dok lokalna pretraga ne uspeva gotovo nikada.

Instanca `1a03` se u rezultatima izdvaja. Naime, iako je istih dimenzija kao `1a02` i `1a04`, sve metode je rešavaju znatno teže. Uzrok nije utvrđen u okviru ovog rada.

Na najvećim instancama, `ft10` i `ft20`, nijedna metoda ne dostiže objavljeni optimum. Ipak, simulirano kaljenje je i tu najbliže, sa prosečnim odstupanjem od 3,8% odnosno 1,6%.

Posmatrano na svim instancama, prosečno odstupanje od optimuma iznosi 0,86% za kaljenje, 2,26% za promenljive okoline, 2,68% za genetski algoritam i 4,19% za lokalnu pretragu sa ponovnim pokretanjem. Poredak je isti u sve tri tabele, što ga čini pouzdanim zaključkom.

3.5 Egzaktno rešavanje

Radi provere ispravnosti razvijenih metoda i utvrđivanja stvarnih optimuma, problem je rešen i egzaktno, celobrojn timer linearnim programiranjem. Korišćena je disjunktivna formulacija (Manne 1960), opisana u odeljku o celobrojn timer modelu, dok je model rešen putem rešavača CPLEX. Pritom, svakoo pokretanje vremenski je unapred ograničeno na jedan sat i zahtevana je stroga optimalnost. Rezultati egzaktnog rešavanja dati su u sledećoj tabeli:

Tabela 3.6: Rezultati egzaktnog rešavanja. Podebljane vrednosti su dokazano optimalne.

instanca	promenljivih	ograničenja	rešenje	donja granica	procep	vreme [s]
mini3	19	27	11	11	0	0,1
ft06	127	216	55	55	0	0,3
1a01	276	500	666	666	0	5,2

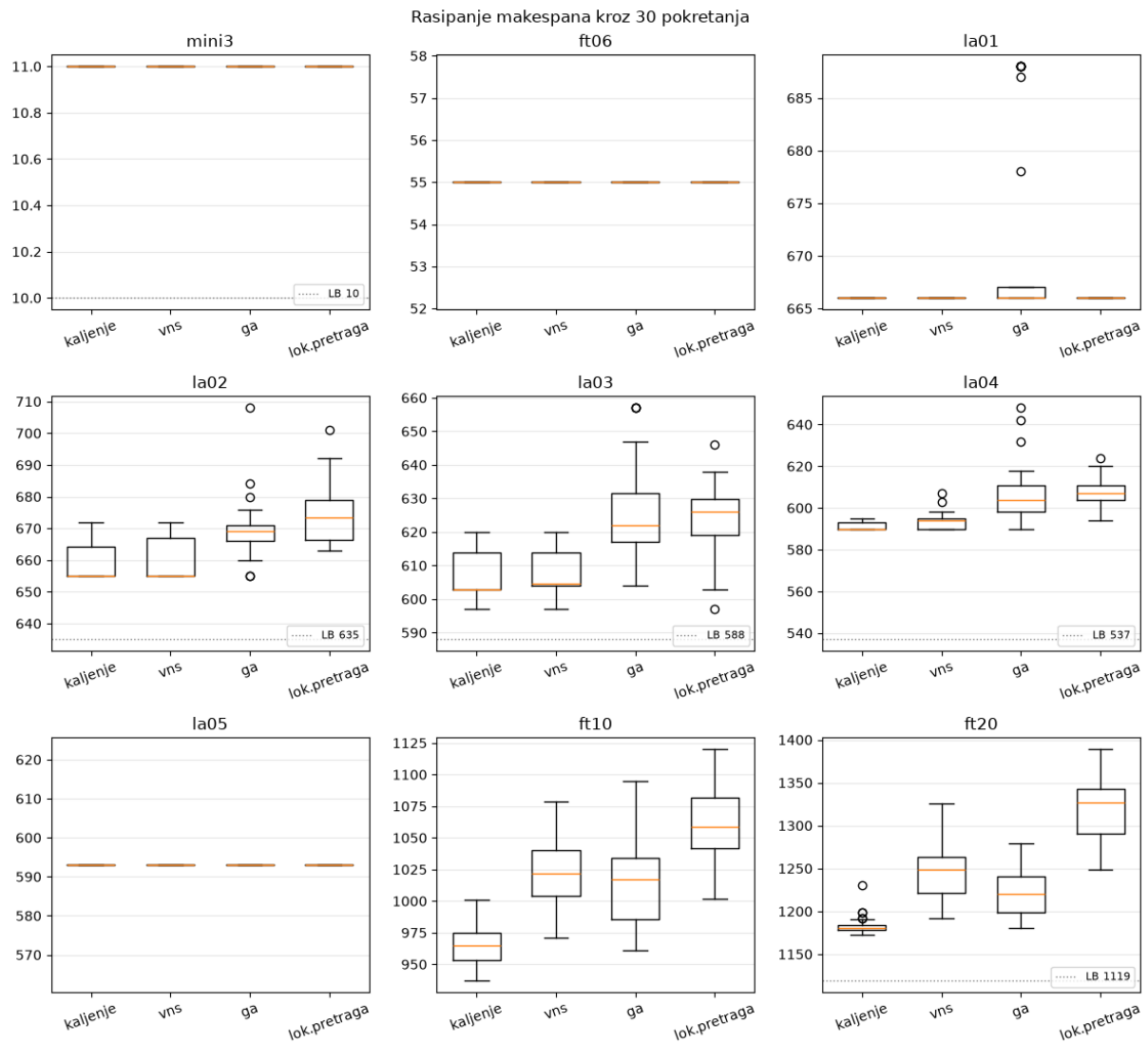
instanca	promenljivih	ograničenja	rešenje	donja granica	procep	vreme [s]
1a02	276	500	655	655	0	3,0
1a03	276	500	597	597	0	2,7
1a04	276	500	590	590	0	1,6
1a05	276	500	593	593	0	9,8
ft10	551	1000	930	930	0	50,8
ft20	1051	2000	1182	657	44,4 %	> 3600

Osam od devet instanci rešeno je do dokazane optimalnosti i sve dobijene vrednosti poklapaju se sa objavljenim optimumima iz tabele 3.2. Ovime je nezavisno potvrđeno da implementirani dekodier ne proizvodi neizvodljive rasporede.

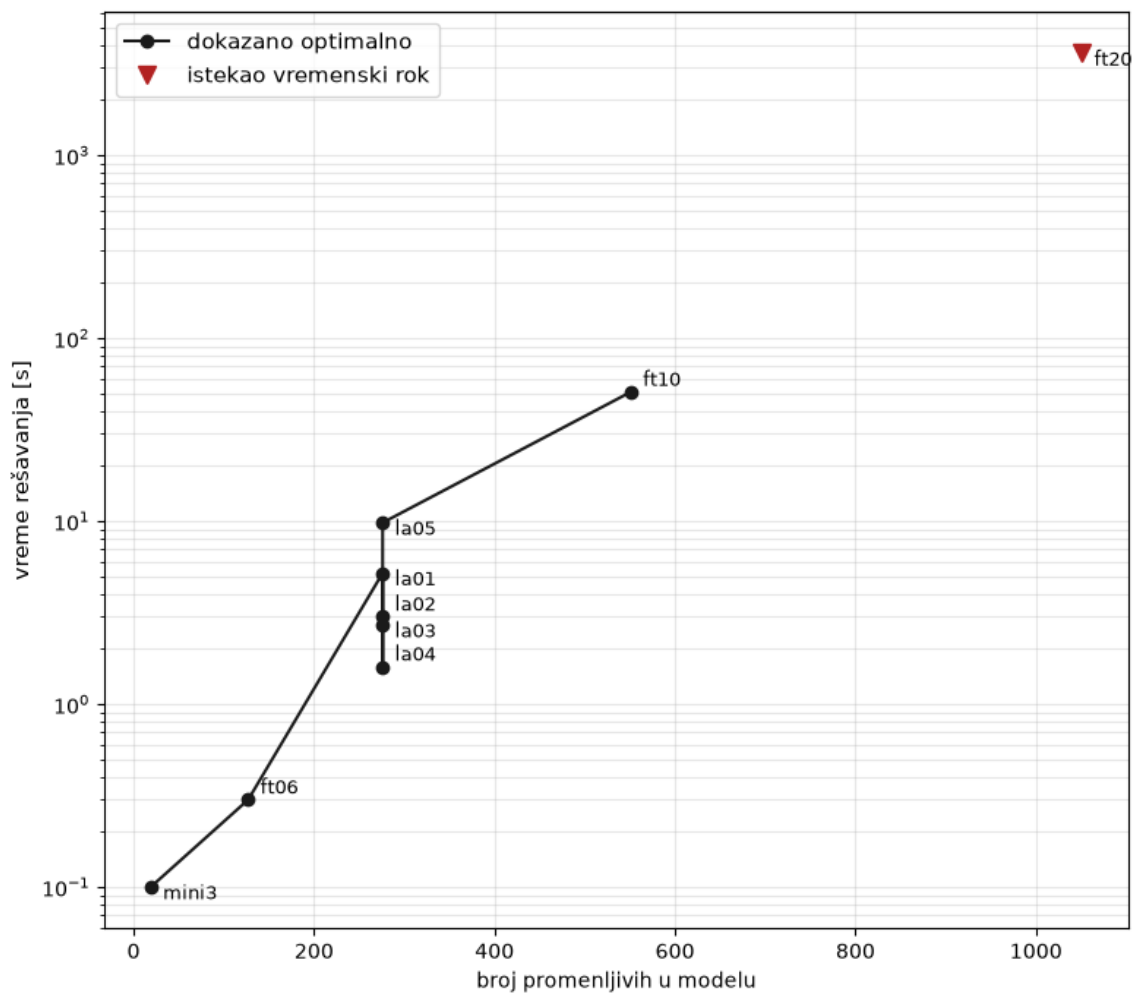
Primetimo da vreme rešavanja raste izrazito brzo sa veličinom instance: 0,3 sekunde za **ft06**, oko 3 sekunde za instance **1a01–1a05**, 50,8 sekundi za **ft10**, dok **ft20** nije rešen ni nakon sat vremena. Upravo ovde vidimo najveću vrednost metaheuristika.

Posvetimo na kratko pažnju instanci **ft20**. Nakon sat vremena rešavač je pronašao rešenje vrednosti 1182 i dokazao donju granicu 657. U pitanju je razilaženje od preko 44%. Poređenja radi, simulirano kaljenje je u istom okruženju pronašlo rešenje vrednosti 1173 za oko pet sekundi. Rezultat je da je primenom metode optimizacije putem metaheuristika pronađeno **bolje rešenje u vremenu manjem za tri reda veličine**.

Pored toga, jednostavna donja granica opisana u odeljku o donjim granicama iznosi za **ft20** **1119**, što je znatno jače ograničenje od granice 657 koju je rešavač dokazao za sat vremena. Kombinovanjem te granice sa najboljim pronađenim rešenjem dobija se interval [1119, 1173], odnosno raskol od 4,6% umesto 44,37%.



Slika 3.2: Rasipanje vrednosti funkcije cilja kroz 30 pokretanja, po instanci i metodi. Kutija obuhvata srednjih 50 % rezultata, linija u njoj je medijana, a kružići označavaju odudarajuće vrednosti.



Slika 3.3: Vreme rešavanja celobrojnog modela u zavisnosti od broja promenljivih. Vertikalna osa je logaritamska. Instanca **ft20** nije rešena u zadatom roku od sat vremena, pa njena vrednost predstavlja donju procenu potrebnog vremena.

Poglavlje 4

Zaključak

Poglavlje 5

Literatura

- Beasley, J. E. 1990. „OR-Library: Distributing Test Problems by Electronic Mail“. *Journal of the Operational Research Society* 41 (11): 1069–72.
- Fisher, H., i G. L. Thompson. 1963. „Probabilistic Learning Combinations of Local Job-Shop Scheduling Rules“. U *Industrial Scheduling*, uredio J. F. Muth i G. L. Thompson, 225–51. Englewood Cliffs, New Jersey: Prentice Hall.
- Jain, A. S., i S. Meeran. 1999. „Deterministic Job-Shop Scheduling: Past, Present and Future“. *European Journal of Operational Research* 113 (2): 390–434.
- Lawrence, S. 1984. „Resource Constrained Project Scheduling: An Experimental Investigation of Heuristic Scheduling Techniques (Supplement)“. Pittsburgh, Pennsylvania: Graduate School of Industrial Administration, Carnegie-Mellon University.
- Manne, Alan S. 1960. „On the Job-Shop Scheduling Problem“. *Operations Research* 8 (2): 219–23.